



Informe Final de Proyecto

Máster en Inteligencia Artificial, Cloud Computing y DevOps

Autores:

Sergio Soriano

Guillermo Parés

Módulo 5 - Machine Learning y Deep Learning

Mayo 2026

Índice

1. Definición de roles.....	3
2. Justificación del problema.....	3
3. Análisis exploratorio de datos (EDA).....	4
3.1 Descripción de dataset.....	4
3.2 Variable objetivo.....	4
3.3 Valores nulos y duplicados.....	5
3.4 Variables con leakage.....	6
3.5 Estadísticas descriptivas de variables numéricas.....	6
3.6 Variables categóricas.....	7
4. Diseño del sistema.....	7
4.1 Pipeline de entrenamiento.....	8
4.2 Módulos principales.....	8
4.2.1 Data loader.....	8
4.2.2 Model trainer.....	8
4.2.3 Evaluator.....	9
4.2.4 Predictor.....	9
4.3 API REST con Fast API.....	9
4.4 Interfaz visual con Streamlit.....	9
4.5 Registro de experimentos con ML Flow.....	10
4.6 Estructura del repositorio.....	11
5. Resultado y elección final.....	11
5.1 Comparativa de modelos.....	11
5.2 Curvas ROC comparativas.....	12
5.3 Matriz de confusión - Random Forest.....	13
5.4 Importancia de variables.....	14
5.5 Modelo seleccionado: Random Forest.....	14
6. Reflexión crítica sobre limitaciones y mejoras.....	15
6.1 Limitaciones.....	15
6.2 Mejoras.....	15
6.3 Valoración global.....	15

1. Definición de roles

Tarea	Responsable
EDA y limpieza de datos	Guillermo Parés
Pipeline de preprocesamiento	Sergio Soriano
Implementación modelos ML	Sergio Soriano
Red neuronal Keras	Guillermo Parés
Evaluación y visualización	Sergio Soriano
MLFlow Tracking	Sergio Soriano
FastAPI + Streamlit	Sergio Soriano
Docker / CI	Sergio Soriano
Informe y documentación	Guillermo Parés

2. Justificación del problema

Las cancelaciones de reservas hoteleras representan uno de los mayores retos operativos del sector. Cuando un cliente cancela, el hotel pierde ingresos que ya había planificado y, si la cancelación es tardía, difícilmente puede volver a vender esa habitación. En paralelo, el overbooking como estrategia defensiva genera experiencias negativas si no se calibra bien.

Predecir con antelación si una reserva va a cancelarse permite al hotel tomar decisiones proactivas:

- Ajustar la política de overbooking de forma inteligente.
- Priorizar esfuerzos comerciales en reservas de alto riesgo.
- Optimizar la gestión de inventario y precios dinámicos.

Se utiliza AUC-ROC como métrica principal porque:

1. El problema de cancelación de reservas suele tener desbalance de clases.
2. AUC-ROC mide la capacidad de separación entre clases para todos los umbrales.
3. Es menos sensible a desbalance que accuracy como métrica única.

Métricas secundarias

- Accuracy: referencia global, útil pero puede ser engañosa con desbalance.
- Precision: controla falsos positivos (predecir cancelación cuando no ocurre).
- Recall: controla falsos negativos (no detectar cancelaciones reales).
- F1-score: equilibrio entre precisión y recall.

3. Análisis exploratorio de datos (EDA)

El análisis exploratorio se realizó sobre el dataset “**dataset_practica_final.csv**” antes de aplicar ninguna transformación. El objetivo fue entender la estructura de los datos, detectar problemas de calidad y fundamentar las decisiones de preprocesamiento.

3.1 Descripción de dataset

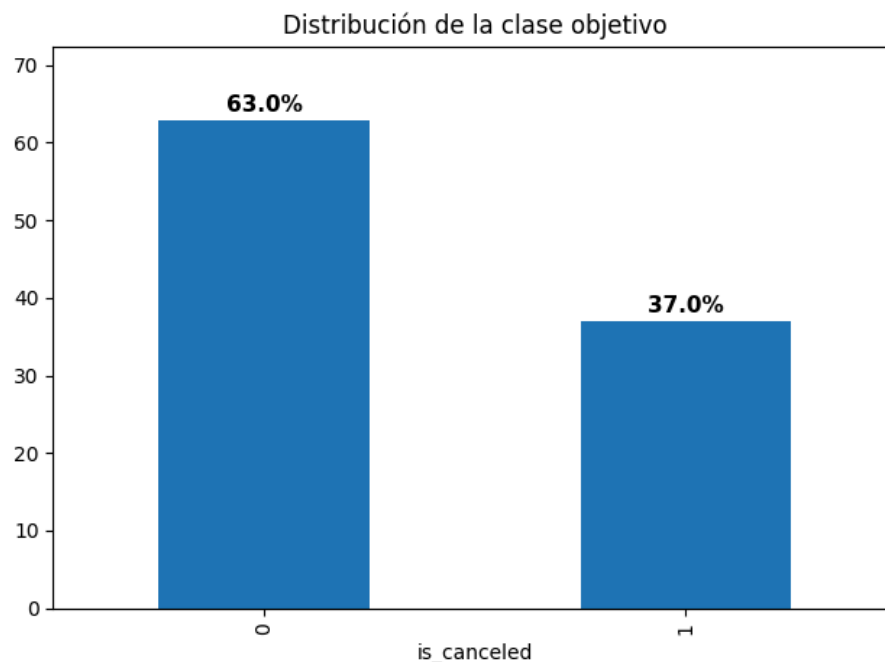
El dataset contiene 119.390 registros y 32 columnas, recogiendo reservas de dos tipos de hoteles (City Hotel y Resort Hotel) entre los años 2015 y 2017. Cada fila representa una reserva individual, con información sobre las características de la estancia, el perfil del cliente y el canal de distribución.

Dimensión	Valor
Filas	119.390
Columnas	32
Variables numéricas	16
Variables categóricas	12
Variable objetivo	“is_canceled” (binaria: 0 / 1)
Periodo temporal	2015 - 2017

3.2 Variable objetivo

La variable objetivo presenta un desbalance moderado. Con una proporción 63/37, el dataset no está críticamente desbalanceado, pero sí lo suficiente como para que el accuracy no sea una métrica fiable por sí sola. Un modelo que predice siempre “no cancelada” alcanzaría ya un 63% de acierto sin aprender nada. Por este motivo

se eligió AUC-ROC como métrica principal, ya que evalúa la capacidad de separación del modelo para todos los umbrales posibles, siendo más robusta ante el desbalance.



3.3 Valores nulos y duplicados

Se identificaron cuatro columnas con valores nulos.

Columna	Nulos	% total	Transformación
children	4	0,003%	Imputar con 0
country	488	0,41%	Imputar con "Unknown"
agent	16.340	13,69%	Transformada en variable binaria "has_agent"
company	112.593	94,30%	Eliminada (prácticamente vacía)

Se detectaron **31.994 filas duplicadas (26.8%)**. Se entiende que estos registros duplicados son errores de imputación de datos ya que es raro que dos personas compartan la misma reserva, habitación, fechas, precio, estado de reserva etcétera (se suele hacer una reserva por persona, pareja o grupo de clientes).

Estas filas son eliminadas usando la función **drop_duplicates()**.

3.4 Variables con leakage

Se detectaron dos columnas que causan fuga de información (data leakage) y que se eliminaron antes del entrenamiento:

- **reservation_status:** Contiene directamente si la reserva fue "Canceled", "Check-Out" o "No-Show". Es el target recodificado.
- **reservation_status_date:** Fecha en que cambió el estado de la reserva. Solo existe si ya se produjo la cancelación.

Incluir estas columnas haría que el modelo "hiciera trampa": aprendería a predecir la cancelación a partir de información que solo está disponible después de que el evento ocurra.

3.5 Estadísticas descriptivas de variables numéricas

Unas de las variables numéricas más relevantes muestra los siguientes comportamientos:

Variable	Media	Mediana	Std	Max
lead_time	104 días	69 días	107	737 días
adr	101,8 €	94,6 €	50, 5 €	5.400 €
total_of_special_requests	0,57	0	0,79	5
previous_cancellations	0,09	0	0,84	26
required_car_parking_spaces	0,06	0	0,24	8

- **lead_time:** Cuanta más antelación de la reserva, más probable es que se cancele.
- **adr:** El precio/noche es importante porque las reservas baratas tienden a cancelarse más ya que supone un compromiso económico menor
- **total_of_special_requests:** Aunque la mayoría de reservas tienen cero peticiones, los clientes que piden algo especial es más probable que no cancelen ya que ya están planificando su estancia.
- **previous_cancellations:** El historial de cancelaciones es el predictor más

directo, aunque la mediana cero, los pocos clientes que ya tienen cancelaciones en su historial son muy relevantes.

- **required_car_parking_spaces:** Este caso es muy similar al de las peticiones. Aunque la media sea muy baja, clientes que muestran interés y gastan más en cosas como parking suelen ser clientes que ya están planificando su viaje.

Además, en la información descriptiva de “adr” se detecta que su valor mínimo es -6,38€. Al tratarse de un campo que muestra el precio noche es algo imposible y seguramente se trate de un error al introducir los datos, para asegurarnos de que no influya en el entrenamiento nos encargamos de eliminarlo.

3.6 Variables categóricas

El análisis de las variables categóricas se centra en aquellas que presentan mayor variación en la tasa de cancelación entre sus categorías, ya que son las que más información aportan al modelo.

- **Tipo de hotel (hotel):** El dataset recoge reservas de dos tipos de establecimiento: City Hotel (79.330 reservas, 61% del total) y Resort Hotel (40.060 reservas, 39%). La tasa de cancelación difiere de forma notable entre ambos: el hotel de ciudad presenta una tasa aproximada del 41%, frente al 28% del hotel de resort.
- **Segmento de mercado (market_segment):** El canal Online TA concentra el 79,10% de todas las reservas. Reservar directamente con el hotel implica mayor intención y, en muchos casos, mayor fidelidad del cliente. Esta variable es uno de los predictores categóricos más informativos del dataset.

Entre ellas también se encuentran variables como customer_type, deposit_type y meal.

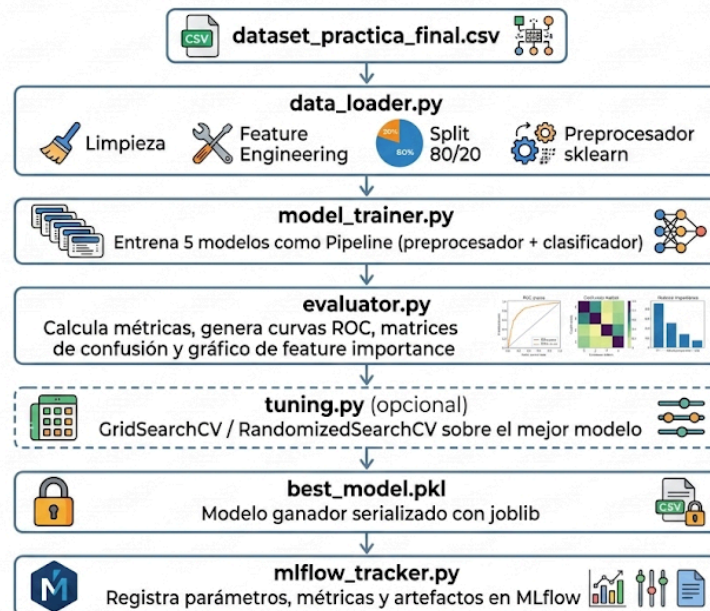
4. Diseño del sistema

El sistema se ha diseñado siguiendo una arquitectura modular donde cada componente tiene una responsabilidad única y bien definida. El objetivo es que el flujo completo, desde la carga de datos hasta la inferencia, pueda ejecutarse con un único comando y sea fácilmente extensible.

4.1 Pipeline de entrenamiento

El flujo principal se orquesta desde `trainer.py`, que actúa como punto de entrada y encadena todos los módulos en orden:

FLUJO DE ENTRENAMIENTO DE MACHINE LEARNING (ESQUEMA)



4.2 Módulos principales

4.2.1 Data loader

Corresponde al fichero `data_loader.py`. Responsable de toda la preparación del dato. Carga el CSV, aplica las transformaciones definidas en el EDA (imputación de nulos, conversión de `agent` a `has_agent`, agrupación de países, eliminación de columnas con leakage) y construye el preprocesador de sklearn mediante un `ColumnTransformer` con dos ramas: una numérica (`SimpleImputer` + `StandardScaler`) y una categórica (`SimpleImputer` + `OneHotEncoder`). Devuelve un objeto `DataBundle` con los conjuntos de train y test ya separados.

Este preprocesamiento se construye mediante la función `build_preprocessor()` que se encuentra en la misma función, detecta automáticamente las columnas numéricas y categóricas del dataset y aplica una transformación distinta a cada tipo:

- **Variables numéricas:** se imputan valores nulos con la mediana y se escalan con `StandardScaler` para que todas las variables queden en la misma escala, esto último es especialmente importante para la regresión logística y la red neuronal.

- **Variables categóricas:** los nulos se imputan con el valor más frecuente y se aplica OneHotEncoder para convertir cada categoría en una columna binaria.

4.2.2 Model trainer

Corresponde al fichero `model_trainer.py`. Define y entrena los cinco modelos del proyecto. Cada modelo se construye como un Pipeline de sklearn que encadena el preprocesador de DataBundle con el clasificador correspondiente. Esto garantiza que el preprocesamiento forme parte del modelo y no haya fuga de información entre train y test. Incluye un wrapper propio (KerasClassifier) para integrar la red neuronal de TensorFlow dentro del ecosistema de sklearn. Los modelos entrenados se serializan con joblib en la carpeta `models/`.

4.2.3 Evaluator

Corresponde al fichero `evaluator.py`. Recibe los modelos entrenados y el conjunto de test y calcula las cinco métricas del proyecto (accuracy, precision, recall, F1-score y AUC-ROC) para cada uno. Genera automáticamente tres visualizaciones: curvas ROC comparativas, matrices de confusión individuales y el gráfico de importancia de variables del mejor modelo. Todos los resultados se guardan en `outputs/`.

4.2.4 Predictor

Corresponde al fichero `predictor.py`. Módulo de inferencia. Carga el `best_model.pkl` y expone una función que recibe un diccionario con las características de una reserva, construye el DataFrame correspondiente y devuelve la predicción junto con la probabilidad de cancelación. Es el módulo que consume la API.

4.3 API REST con Fast API

La API se define en `src/api.py` y permite consumir el sistema de forma programática sin necesidad de ejecutar código Python directamente. Expone los siguientes endpoints:

Endpoint	Método	Descripción
/health	GET	Comprueba que el servicio esté activo
/train	POST	Lanza el pipeline completo de entrenamiento
/predict	POST	Predice si una reserva se cancelará
/evaluate	GET	Devuelve las métricas del último entrenamiento

/runs	GET	Historial de ejecuciones almacenado en SQLite
-------	-----	---

4.4 Interfaz visual con Streamlit

La aplicación app.py ofrece una interfaz web accesible en <http://localhost:8501> que permite interactuar con el sistema sin usar la línea de comandos. Está organizada en tres secciones: entrenar modelos (delegando la llamada a la API), visualizar los resultados y métricas del último entrenamiento, y realizar predicciones individuales introduciendo los datos de una reserva mediante un formulario.

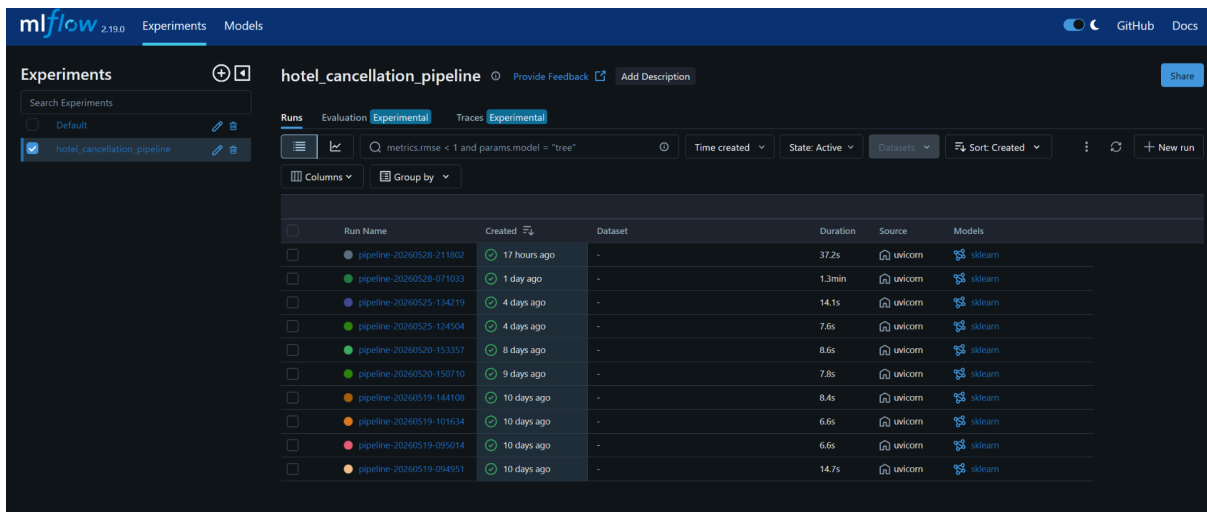
The screenshot shows a web application titled "Proyecto Final ML - Hotel Booking Cancellation". On the left, a sidebar contains three radio buttons: "Entrenar" (selected), "Evaluar", and "Predecir". The main content area is titled "Entrenamiento" and includes several configuration options:

- ☒ Quick mode
- ☐ Saltar red neuronal
- ☐ Activar tuning (Grid/Randomized)
- Método de tuning: randomized (selected in a dropdown menu)
- Folds CV: A slider set to 3, with a range from 2 to 5.
- Iteraciones Randomized: A slider set to 15, with a range from 5 to 48.
- ☒ Activar tracking MLflow

 At the bottom of the configuration section is a red button labeled "Ejecutar entrenamiento".

4.5 Registro de experimentos con ML Flow

MLflow se utiliza para registrar de forma automática cada ejecución del pipeline. Por cada entrenamiento se almacenan los hiperparámetros de cada modelo, las métricas finales de evaluación, los artefactos generados (CSV de métricas, gráficos PNG) y el modelo ganador con su firma de entrada y salida. En el entorno Docker, MLflow corre como servicio independiente con backend en PostgreSQL y su interfaz es accesible en <http://localhost:5000>, permitiendo comparar visualmente distintas ejecuciones a lo largo del tiempo.



4.6 Estructura del repositorio

data/raw/	# Dataset original (solo lectura)
docs/	# Informe final y figuras
models/	# Modelos serializados (.pkl)
notebooks/	
exploracion/	# EDA y pruebas exploratorias
finales/	# Comparativa final de modelos
outputs/	# Métricas CSV, gráficos PNG, runs.db
src/	
config.py	# Rutas y constantes centralizadas
data_loader.py	# Carga y preprocesamiento
model_trainer.py	# Entrenamiento de modelos
evaluator.py	# Métricas y visualizaciones
predictor.py	# Inferencia
tuning.py	# Optimización de hiperparámetros
mlflow_tracker.py	# Registro de experimentos
api.py	# API REST (FastAPI)
api_models.py	# Modelos Pydantic para la API
app.py	# Interfaz Streamlit
trainer.py	# Script principal del pipeline
docker-compose.yml	# Orquestación de servicios
requirements.txt	# Dependencias del proyecto

5. Resultado y elección final

5.1 Comparativa de modelos

El pipeline se entrenó sobre 95.512 reservas (80% del dataset) y se evaluó sobre las 23.878 restantes (20%), con semilla fija random_state=42 para garantizar la reproducibilidad. La métrica principal de selección es AUC-ROC.

Model	Accuracy	Precision	Recall	F1 Score	ROC AUC
random_forest	0.890401	0.885778	0.808366	0.845304	0.955935
neural_network	0.866949	0.843099	0.787337	0.814265	0.944030
gradient_boosting	0.854134	0.843806	0.743923	0.790723	0.933085
decision_tree	0.825865	0.790649	0.720746	0.754081	0.906324
logistic_regression	0.815772	0.810996	0.655399	0.724942	0.894911

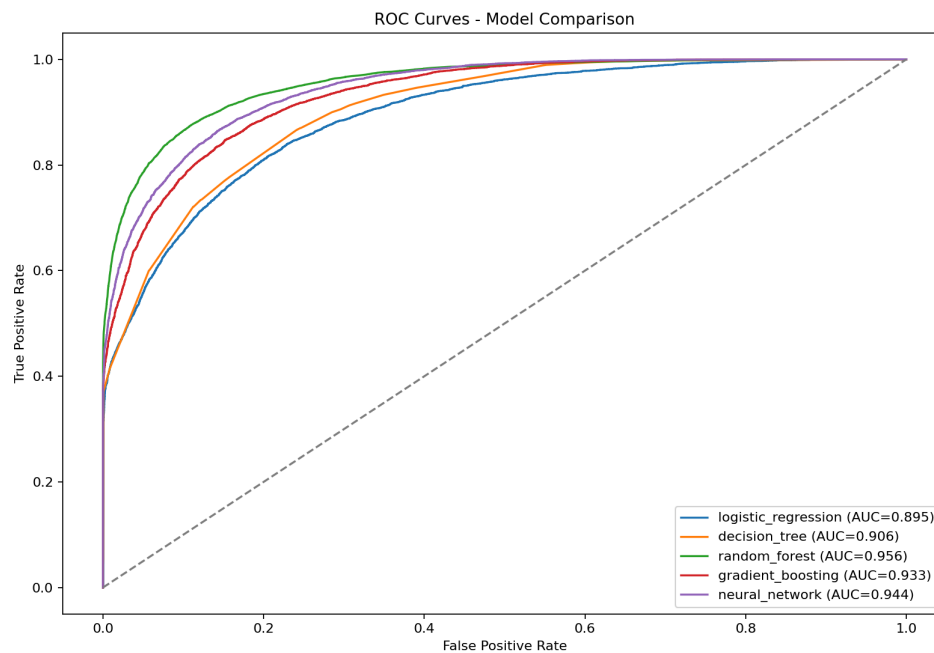
Random Forest lidera en todas las métricas de forma simultánea, lo que confirma que su ventaja no es puntual sino consistente independientemente del criterio de evaluación utilizado.

5.2 Curvas ROC comparativas

Las curvas ROC permiten visualizar la capacidad discriminativa de cada modelo para todos los umbrales de clasificación posibles. El área bajo la curva (AUC)

resume esta capacidad en un único valor: cuanto más se acerca a 1, mejor separa el modelo de las reservas canceladas de las no canceladas.

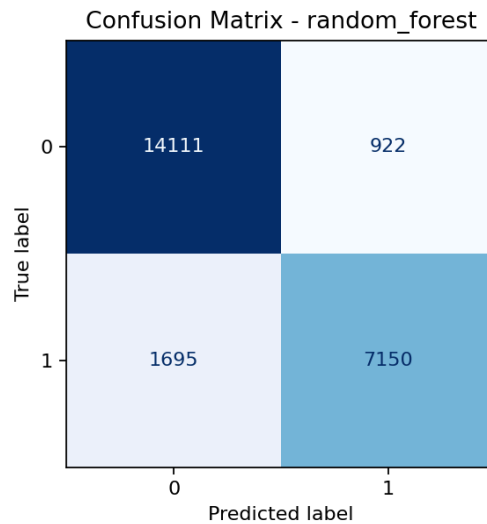
Random Forest y la red neuronal se separan claramente del resto, con curvas que se aproximan más a la esquina superior izquierda del gráfico. La Regresión Logística, pese a ser el modelo más sencillo, muestra un AUC de 0.896, lo que indica que incluso un modelo lineal captura una señal predictiva relevante en los datos.



5.3 Matriz de confusión - Random Forest

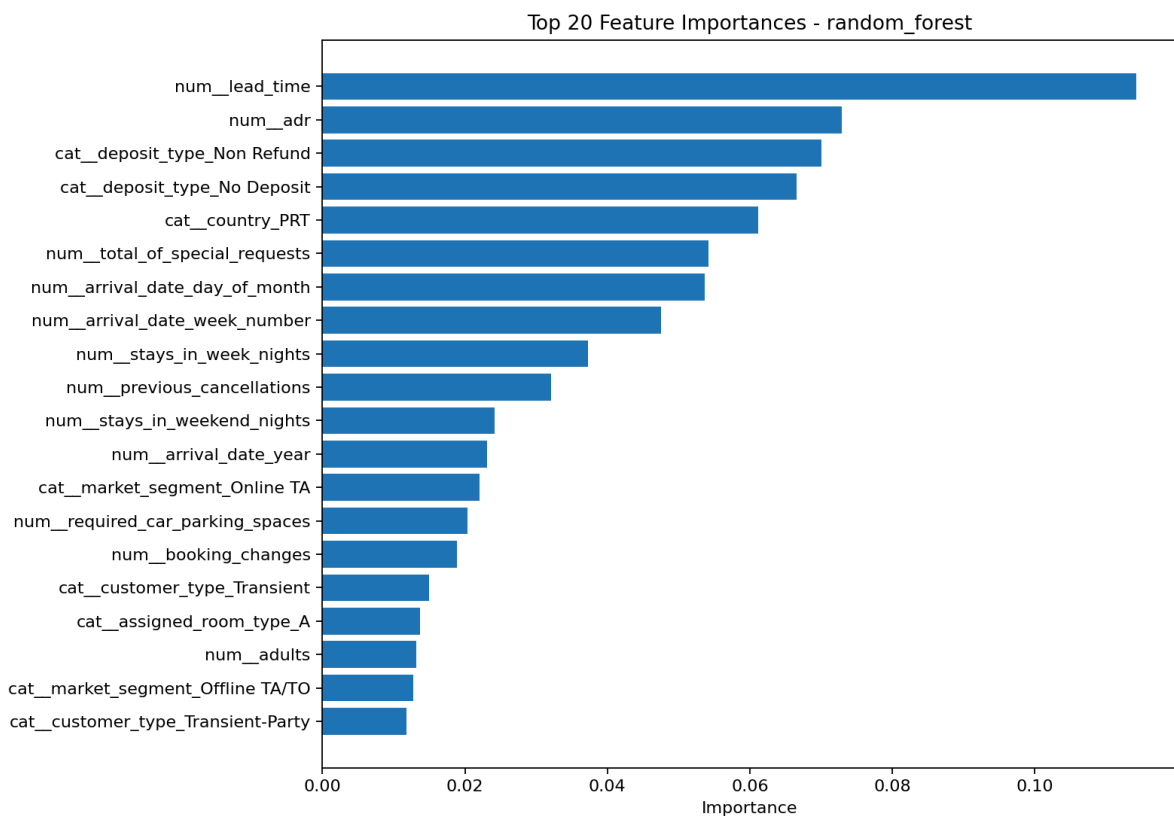
Viendo la matriz de confusión podemos sacar las siguientes conclusiones sobre las predicciones.

- Hay 14.111 reservas que NO se han cancelado y que el modelo ha predicho correctamente que NO (True Negatives)
- Hay 922 reservas que NO se cancelaron pero el modelo predijo que SÍ (False Positives)
- 1.695 reservas que SÍ se cancelaron pero el modelo dijo que NO (False Negatives)
- 7.150 reservas que SÍ se han cancelado y el predijo correctamente que SÍ (True Positives)



5.4 Importancia de variables

Gracias al siguiente gráfico podemos ver las veinte variables más importantes y con mayor poder predictivo en orden descendente.



5.5 Modelo seleccionado: Random Forest

Random Forest se consolida como el mejor modelo del proyecto con un AUC-ROC de 0.959.

La red neuronal queda segunda con AUC 0.945, lo que confirma que existe complejidad suficiente en el problema como para que el Deep Learning añada valor. Sin embargo, con 119.390 registros tabulares y features estructuradas, los modelos basados en árboles siguen siendo más eficientes y más fáciles de interpretar que las redes neuronales, que generalmente destacan con datos no estructurados como imágenes o texto.

6. Reflexión crítica sobre limitaciones y mejoras

Un análisis crítico del trabajo realizado nos permite identificar limitaciones que conviene reconocer y que marcan el camino de iteraciones futuras.

6.1 Limitaciones

- Corte temporal del dataset: Los datos son de 2015-2017 y el split train/test es aleatorio, no temporal. Esto puede generar una estimación optimista del rendimiento real. Además, los patrones de cancelación han cambiado significativamente tras la pandemia, lo que puede haber degradado la capacidad predictiva del modelo sobre reservas actuales.

6.2 Mejoras

- Usar un split temporal (train hasta 2016, test con 2017) para una evaluación más realista
- Recopilar datos más recientes que reflejen el comportamiento post-pandemia

6.3 Valoración global

Las limitaciones identificadas no invalidan el trabajo, sino que reflejan decisiones de alcance tomadas conscientemente. El proyecto sienta una base sólida y modular sobre la que aplicar estas mejoras de forma incremental en un entorno profesional real.